

Trabajo Práctico N° 3

Optimización de Consultas, Índices y Vistas

Para este Trabajo Práctico, el servidor PostgreSQL ha sido configurado intencionalmente para desactivar el procesamiento paralelo (`max_parallel_workers_per_gather = 0`).

En un servidor de producción real con miles de usuarios conectados, tu consulta no tendrá el privilegio de usar todos los núcleos del procesador de forma exclusiva; solo se le asignará 1 hilo de ejecución. Por lo tanto, si tu consulta no usa índices y debe leer todos los registros, sentirás la lentitud real de un cuello de botella. Tu misión es optimizar el código para que el motor resuelva el problema sin fuerza bruta.

Objetivo: Actuar como un Administrador de Base de Datos (DBA) o Desarrollador Senior. Deberás analizar consultas ineficientes, diagnosticar su costo de ejecución y aplicar técnicas de optimización lógica y física para mejorar su rendimiento sobre una base de datos de alto volumen (+5 millones de registros).

Metodología: Para cada ejercicio deberás tomar nota de

1. El tiempo de ejecución y el costo de la consulta original (usando EXPLAIN ANALYZE).
2. El código SQL optimizado (la consulta reescrita o los CREATE INDEX / CREATE VIEW necesarios).
3. El nuevo tiempo de ejecución logrado.
4. Una breve justificación técnica de tu mejora.

Ejercicio 1

Escenario: El departamento de Marketing lanzó una campaña en la provincia de Santa Fe y solicitó un reporte de todas las compras realizadas en dicha provincia durante los años 2024 y 2025. Un desarrollador Junior entregó el siguiente código:

```
SELECT c.nombre, p.fecha, p.importe
FROM pedidos p
JOIN clientes c ON p.id_cliente = c.id_cliente
JOIN provincias pr ON c.id_provincia = pr.id_provincia
WHERE pr.nombre = 'Santa Fe' AND (EXTRACT(YEAR FROM p.fecha) = 2024
OR EXTRACT(YEAR FROM p.fecha) = 2025);
```

Al ejecutarla, el servidor consumió casi el 100% de la CPU durante varios segundos, leyendo millones de bloques de disco innecesariamente.

Consignas:

1. Ejecuta el EXPLAIN ANALYZE de la consulta. ¿Cuánto tiempo tardó? ¿Notaste algún Hash Join o Seq Scan masivo?
2. Optimización Lógica: Reescribe el filtro de fecha para que la consulta pueda utilizar índices para acelerar la búsqueda, en lugar de escanear toda la tabla.
3. Optimización Física: Escribe la sentencia para crear un índice en la tabla pedidos que ayude al motor a no tener que escanear los 5 millones de registros.
4. Vuelve a ejecutar el EXPLAIN ANALYZE con tu nueva consulta y tu índice creado. Registra el nuevo tiempo, verifica el impacto de tus cambios y explica brevemente a qué se debe el cambio de rendimiento.

Ejercicio 2

Escenario: Los operarios de la 'Sucursal 15' tienen un monitor en el depósito que muestra permanentemente los pedidos que están en estado 'PENDIENTE', ordenados por fecha para armar los paquetes del más viejo al más nuevo.

```
EXPLAIN ANALYZE
SELECT id_pedido, id_articulo, fecha, cantidad
FROM pedidos
WHERE id_sucursal = 15 AND estado = 'PENDIENTE'
ORDER BY fecha ASC;
```

Aunque la consulta solo devuelve unas pocas decenas de filas, tarda demasiado tiempo en pantalla porque el motor está escaneando toda la tabla.

Consignas:

1. Optimización Física: implementa una optimización física específica para esta consulta en la tabla pedidos.
2. Vuelve a ejecutar el EXPLAIN ANALYZE. Verifica si lograste que el motor use un Index Scan en lugar de un Seq Scan. ¿A cuántos milisegundos bajó el tiempo de respuesta?

Ejercicio 3

Escenario: Frustrado por la lentitud general del sistema, un pasante decidió ejecutar el siguiente bloque de código en producción:

```
CREATE INDEX idx_pedidos_1 ON pedidos(id_cliente);
CREATE INDEX idx_pedidos_2 ON pedidos(id_articulo);
CREATE INDEX idx_pedidos_3 ON pedidos(fecha);
CREATE INDEX idx_pedidos_4 ON pedidos(cantidad);
CREATE INDEX idx_pedidos_5 ON pedidos(importe);
CREATE INDEX idx_pedidos_6 ON pedidos(estado);
```

Consignas:

1. Basándote en la teoría, explica detalladamente qué impacto gravísimo tendrá esta "sobredosis" de índices simples en el rendimiento general del sistema comercial, específicamente durante el Hot Sale.
2. Ejecuta el siguiente comando y evalúa el rendimiento:

```
EXPLAIN ANALYZE
INSERT INTO pedidos (id_cliente, id_articulo, id_sucursal,
fecha, cantidad, importe, estado)
VALUES
(1, 1, 1, CURRENT_DATE, 1, 1000.00, 'PAGADO'),
(2, 2, 2, CURRENT_DATE, 1, 2000.00, 'PAGADO'),
(3, 3, 3, CURRENT_DATE, 1, 3000.00, 'PAGADO'),
(4, 4, 4, CURRENT_DATE, 1, 4000.00, 'PAGADO'),
(5, 5, 5, CURRENT_DATE, 1, 5000.00, 'PAGADO');
```

3. Escribe los comandos para borrar (DROP INDEX) estos índices innecesarios que saturan el disco.

Ejercicio 4

Escenario: El directorio de la empresa revisa todos los días un tablero general que muestra cuánto se recaudó históricamente por cada categoría de artículo en cada sucursal.

```
SELECT s.nombre AS sucursal, a.categoria, SUM(p.importe) AS
total_recaudado
FROM pedidos p
JOIN sucursales s ON p.id_sucursal = s.id_sucursal
JOIN articulos a ON p.id_articulo = a.id_articulo
GROUP BY s.nombre, a.categoria;
```

Cruzar y agrupar 5 millones de registros y 3 tablas tarda varios segundos. Los directores actualizan el tablero 50 veces por día, ahogando al procesador del servidor. Aceptan que los datos tengan un desfase de hasta 24 horas (no necesitan el dato del último segundo).

Consignas:

1. Crea una Vista Estándar (CREATE VIEW) que encapsule esta consulta. Ejecuta un EXPLAIN ANALYZE SELECT * FROM tu_vista;. ¿Se solucionó el problema de rendimiento? Explica el porqué.
2. Elimina la vista estándar y crea una Vista Materializada (CREATE MATERIALIZED VIEW) con la misma lógica.

3. Ejecuta un `EXPLAIN ANALYZE` llamando a la vista materializada. Compara el costo y el tiempo de ejecución con el del paso 1.
4. Un cliente acaba de comprar una heladera ('Hogar') en la 'Sucursal 1' por \$5,000,000 de pesos. Has insertado el registro en pedidos con el comando: `INSERT INTO pedidos (id_cliente, id_articulo, id_sucursal, fecha, cantidad, importe) VALUES (1, 1, 1, CURRENT_DATE, 1, 5000000.00);`, pero la vista materializada no lo refleja. Escribe el comando exacto que el DBA debe programar en las tareas nocturnas del servidor para que la vista recupere la consistencia con las tablas base.